

Chapter 1

Occupancy Prediction

1.1 Purpose of Occupancy Prediction

Occupancy prediction is the first decision-making stage of the smart hotel application. Its purpose is to estimate the future operating state of a room before a lighting or HVAC recommendation is produced. In the implemented system, the output belongs to one of three classes:

- *Occupied*: a guest is expected to be present in the room;
- *Vacant*: the room is expected to be unoccupied; and
- *Cleaning*: housekeeping activity is expected.

This distinction is important because the appropriate control strategy is different for each state. An occupied room should prioritize comfort, a vacant room can use more aggressive energy-saving settings, and a room under cleaning may require bright lighting and a moderate HVAC setting for hotel staff.

1.2 Data Sources and Prediction Target

The occupancy subsystem combines records from `PIRSensorData.csv` and `hotelReservationData.csv`. The PIR dataset supplies timestamped room activity and room-state information. The reservation dataset provides guest and booking context. The two datasets are joined using the guest identifier when a valid identifier is available.

The classical occupancy model predicts the room state 60 minutes in the future. Because measurements are recorded every five minutes, this corresponds to shifting the target by 12 samples:

$$y_t = \text{room_state}_{t+12}.$$

This target is intentionally different from simply reproducing the current room state. It allows the recommendation system to prepare for a state change, for example by reducing energy use before a room becomes vacant.

1.3 Feature Engineering

The Random Forest training script constructs both instantaneous and historical features. The numerical features are:

- room number;
- current PIR motion value;
- mean PIR motion during the previous hour;

- total number of motion detections during the previous hour;
- number of adults and children;
- total guest count;
- whether an active guest identifier is present;
- hour-of-day and day-of-week cyclic features;
- total nights, total reservation amount and stay duration; and
- price per night.

Nationality and room type are treated as categorical variables. Missing numerical values are replaced with the median, categorical values are replaced with the most frequent value, and categorical variables are one-hot encoded. Numerical values are standardized before they are passed to the classifier.

Time is encoded cyclically because hour 23 and hour 0 are temporally close even though their raw numerical values are far apart:

$$h_{\sin} = \sin\left(\frac{2\pi h}{24}\right), \quad h_{\cos} = \cos\left(\frac{2\pi h}{24}\right),$$

$$d_{\sin} = \sin\left(\frac{2\pi d}{7}\right), \quad d_{\cos} = \cos\left(\frac{2\pi d}{7}\right).$$

1.4 Random Forest Occupancy Model

The classical model is implemented as a scikit-learn pipeline containing a `ColumnTransformer` and a `RandomForestClassifier`. The forest uses 160 trees, a minimum leaf size of 20, balanced subsample class weights, parallel execution and a fixed random seed of 42. Class weighting is necessary because Cleaning observations are much less frequent than Occupied and Vacant observations.

A chronological 80/20 split is used instead of a random split. Earlier observations are used for training and later observations are used for testing. This prevents information from future timestamps from leaking into the training set and gives a more realistic estimate of deployment performance.

1.5 Transformer Occupancy Model

The Transformer implementation uses sequences of 12 five-minute observations, representing one hour of room history, and predicts the state 12 steps later. Each time step contains 17 features:

- PIR motion, adult count, child count and total guest count;
- active-guest status;
- cyclic hour and day-of-week values;
- normalized reservation duration and cost variables; and
- one-hot room-type indicators for Deluxe, Standard, Suite and Unknown.

The model first projects the 17-dimensional input to a 64-dimensional hidden space. A trainable positional embedding is added so the network can distinguish the order of observations. The encoder contains two Transformer layers with four attention heads, a feed-forward dimension of 128, GELU activation and dropout of 0.15. After encoding, the hidden states are normalized and averaged over time. A fully connected classification head produces logits for Cleaning, Occupied and Vacant.

The model is trained with AdamW and weighted cross-entropy loss. Gradient clipping is applied during training to reduce instability. The training set contains 320,000 sequences and the test set contains 80,000 sequences.

1.6 Serving-Time Prediction Workflow

The Flask endpoint `/api/predict_occupancy` accepts a room number, timestamp, model type and optional lookback and prediction-horizon values. The backend queries the `pir_sensor_data` table for records between the lookback start time and requested timestamp. The user interface allows the Random Forest, Transformer or automatic model-selection mode to be chosen.

For a Transformer request, the recent observations are converted into a fixed-length sequence and passed through the saved PyTorch model. A softmax operation converts logits into probabilities. For a Random Forest request, a single engineered feature row is passed to the saved scikit-learn pipeline. Both paths return:

- the predicted class;
- prediction confidence;
- a probability for each occupancy class;
- diagnostic features such as motion rate and latest room state; and
- the model name and prediction horizon.

The backend includes two fallback strategies. If no recent PIR history exists, it returns a conservative default distribution that favors Vacant. If a saved model is unavailable, a rule-based score combines the latest state, active guest status, guest count, motion rate and motion count. These fallbacks keep the API operational, although model-based predictions are preferred.

1.7 Occupancy Results

The Random Forest was evaluated on 1,042,300 test rows and achieved an overall accuracy of 98.9%. The Transformer achieved 98.8% accuracy on 80,000 test sequences. Occupied and Vacant were predicted with high F1-scores by both models. However, Cleaning remained difficult because of its very small support. The Random Forest obtained a Cleaning F1-score of 0.100, while the Transformer improved it to 0.248.

These results show why overall accuracy must be interpreted together with class-level precision, recall and F1-score. A model can obtain high accuracy on an imbalanced dataset while still performing poorly on an operationally important minority class.

Chapter 2

Guest Persona Classification

2.1 Role of Persona Classification

Persona classification represents repeated guest behavior as an interpretable category. The application implements separate persona models for lighting and temperature because brightness preference and thermal preference are not necessarily the same. Persona outputs are not final control commands; they are context variables used by the recommendation models.

2.2 Lighting Persona Categories

The lighting subsystem uses the following classes:

- *Balanced*: moderate and varied lighting use;
- *Routine*: repeated lighting patterns at similar times;
- *StaticBright*: consistently high preferred brightness;
- *StaticDim*: consistently low preferred brightness;
- *NightFocused*: stronger activity during late-night hours;
- *Housekeeping*: bright task-oriented lighting; and
- *Unknown*: insufficient or unsupported history.

The five primary guest personas are learned by the trained classifiers. Housekeeping and Unknown are also recognized by the live recommendation service so that operational and missing-data cases can be handled safely.

2.3 Random Forest Lighting Persona Model

The classical lighting persona script groups lighting records by room and date, producing one feature row for each room-day. Only records with an active reservation are used. The target is the dominant `lightning_persona` value for that room-day.

The aggregated features include:

- mean, standard deviation and maximum lighting level;
- percentage of records in which a lamp is on;
- PIR motion rate;

- mean and maximum occupant count;
- mean number of active actors;
- average lighting value for each hour of the day; and
- the proportional usage of each lamp location.

Missing values are imputed with the median and the numerical features are standardized. The Random Forest contains 200 trees, uses a minimum leaf size of five and applies balanced subsample class weighting. A stratified 75/25 split preserves class proportions when each class has sufficient examples.

2.4 Transformer Lighting Persona Model

The Transformer treats each room-day as a complete sequence of 288 five-minute time slots. At every slot, the model receives 19 features: the normalized level of each of the 11 lamp locations, PIR motion, number of occupants, active actors, three behavioral indicators, and sine/cosine hour encodings.

Missing time slots are reindexed and filled with zero. This creates a regular daily sequence even if a lamp has no recorded activity during some periods. Brightness values are clipped to the operational range of 0–80 and normalized by dividing by 80.

The model projects the input into a 96-dimensional representation. It uses three Transformer encoder layers, four attention heads, a feed-forward dimension of 192, GELU activation and dropout of 0.15. After layer normalization, mean pooling summarizes the full day. A classification head maps the pooled representation to persona logits.

Training uses AdamW, class-weighted cross-entropy and gradient clipping. The saved model was trained on 9,132 room-day samples, of which 7,305 were used for training and 1,827 for testing.

2.5 Live Lighting Persona Prediction

The endpoint `/api/predict_lighting_persona` retrieves recent records from the `lightning_data` table. The default history window is 24 hours, while the user interface also supports shorter three-hour and six-hour contexts.

Before prediction, the backend calculates diagnostics that are displayed in the interface:

- mean level while lamps are on;
- proportion of lamp records with a level greater than zero;
- mean nighttime and daytime levels;
- nighttime share of total day/night brightness; and
- mean number of active actors.

The Random Forest path constructs the same room-day style feature row used during training. The Transformer path builds a 288-step sequence aligned to the requested timestamp. Both return class probabilities and confidence. If no trained model is available, a rule-based fallback scores personas using recent mean brightness, lighting ratio, night share and active-actor information.

2.6 Temperature Persona Categories

Temperature behavior is classified into:

- *AlwaysOnComfort*: prioritizes a narrow comfort range;
- *EnergySaver*: accepts wider temperature variation;

- *Reactive*: changes settings after conditions become uncomfortable;
- *Preconditioning*: prepares the room before occupancy; and
- *Housekeeping*: represents staff-oriented HVAC operation.

2.7 Temperature Persona Transformer

The temperature persona model uses 24 sequential observations. Because the temperature dataset is sampled every five minutes, the sequence represents two hours of recent room behavior. The input contains 42 encoded features, including:

- scaled floor, room size, outside temperature, room temperature, setpoint and ideal temperature;
- temperature error and comfort error;
- PIR motion and guest-presence indicators;
- cyclic hour and day-of-week features;
- room facade and room type;
- HVAC mode, occupant state and PIR persona; and
- current room state.

The model uses a 96-dimensional hidden representation, four attention heads, two encoder layers, a feed-forward dimension of 192 and dropout of 0.15. Training uses class-weighted cross-entropy, AdamW and a chronological train/validation/test split. From 500,000 generated sequences, 400,000 were used for training, 50,000 for validation and 50,000 for testing.

2.8 Persona Classification Results

The Random Forest lighting persona classifier achieved 88.5% accuracy and a weighted F1-score of 0.885. The Transformer improved the lighting persona accuracy to 92.5%, with a macro F1-score of 0.926. StaticDim achieved the highest Transformer F1-score, 0.952.

The temperature persona Transformer achieved 80.55% accuracy and a weighted F1-score of 0.8088. AlwaysOnComfort achieved an F1-score of 0.9006, while EnergySaver and Reactive were more frequently confused. Housekeeping and Preconditioning had very high scores but very small support, so those results should not be interpreted as evidence of broad generalization.

Chapter 3

Lighting Recommendation System

3.1 Recommendation Pipeline

The lighting recommendation system is implemented as a chained pipeline:

1. obtain the room number, timestamp and history window;
2. predict future occupancy;
3. predict the guest's lighting persona;
4. retrieve recent room and per-lamp history;
5. generate one recommendation for each known lamp;
6. enforce hardware and operational constraints; and
7. convert current and recommended levels into energy estimates.

The Flask endpoint `/api/lightning_recommendation` can receive occupancy and persona values directly, or it can generate them automatically when `use_model_predictions` is enabled. In automatic mode, the occupancy model uses a one-hour lookback and a 60-minute horizon. The persona model uses the selected lighting history window.

3.2 Lamp Classification

The implementation distinguishes between dimmable LEDs and non-dimmable bulbs. The dimmable group contains:

- `bed_left`, `bed_right`, `dinner_table`, `table` and `hidden_top`.

The non-dimmable group contains:

- `closet`, `corridor_left`, `corridor_right`, `shower`, `cabinet` and `sink`.

This classification affects both recommendation and energy calculation. Dimmable lamps may receive a continuous level between 0 and 80. Non-dimmable lamps are constrained to either 0 or 80.

3.3 Historical Context

For each lamp, the backend obtains the latest level and the mean values over the previous one hour, three hours and full lookback period. Room-wide context includes floor, PIR motion, number of occupants, active actors, reservation status and behavioral indicators such as `hurry_morning`, `lazy_day` and `forgetful`.

The tabular recommendation model additionally uses rolling features:

- per-lamp mean level over 1, 3 and 24 hours;
- per-lamp on-rate over three hours;
- room-wide mean level over 1 and 24 hours; and
- room-wide motion rate over one hour.

All rolling features are shifted by one sample during training. This ensures that a training target is predicted only from earlier observations and avoids using the current target value as historical input.

3.4 Efficient Recommendation Target

The recommendation target is not a direct copy of historical brightness. Instead, the training script constructs an energy-aware target using occupancy, reservation state, recent preference and persona policy.

If the room is Vacant or has no active reservation, the target is zero. If the room is being cleaned, an active lamp is assigned level 80. If a lamp was already off, it remains off. Otherwise, the target is based on the minimum of the current level, recent median preference and persona cap, multiplied by a persona reduction factor:

$$L_{\text{raw}} = \min(L_{\text{current}}, L_{\text{recent}}, C_p)F_p.$$

The result is then limited by the persona’s minimum active level:

$$L_{\text{rec}} = \text{clip}(\max(L_{\text{raw}}, M_p), 0, 80).$$

For example, StaticBright uses a factor of 0.90, a cap of 75 and a minimum active level of 35. StaticDim uses a factor of 0.62, a cap of 35 and a minimum of 10. NightFocused uses a factor of 0.68 and a cap of 45. These policies lower avoidable brightness while retaining differences between guest preferences.

3.5 HistGradientBoosting Recommendation Model

The tabular model uses a `HistGradientBoostingRegressor` with 180 iterations, a learning rate of 0.08, 31 maximum leaf nodes and L2 regularization of 0.01. Numerical variables are median-imputed and standardized. Categorical variables—lamp location, reservation status, occupancy prediction and persona prediction—are imputed and one-hot encoded.

Training uses 10,563,249 rows. The first 80% of the timeline is used for training and the final 20% for testing. Predictions are rounded and clipped to the 0–80 operational range.

3.6 Transformer Recommendation Model

The Transformer regressor uses a sequence of 12 recent five-minute records and predicts the efficient level for the next five-minute step. Each record contains 38 encoded features. These include normalized brightness, PIR motion, occupant counts, behavioral flags, cyclic time values, lamp location, reservation status, occupancy prediction and lighting persona prediction.

The model has a 96-dimensional hidden space, four attention heads, two encoder layers, a feed-forward dimension of 192 and dropout of 0.15. A sinusoidal positional encoding is added to the projected input. The representation of the last sequence token is passed through a regression head. A sigmoid constrains the normalized prediction before it is converted back to the 0–80 level range.

3.7 Serving Constraints and Fallback Behavior

The live backend can select the HistGradientBoosting model, Transformer model or automatic mode. If the selected model files are unavailable, the service uses the explicit persona policy as a rule-based fallback.

Several constraints are applied after model inference:

- a currently inactive lamp remains off;
- a non-dimmable lamp is snapped to either 0 or 80;
- a Vacant room or inactive reservation turns lamps off;
- Cleaning mode keeps required lamps bright; and
- recommendations remain within the valid device range.

These constraints are important because a statistically valid model output may still be physically impossible or operationally inappropriate.

3.8 Recommendation Output

The API returns a record for every lamp containing current level, recommended level, one-, three- and 24-hour means, lamp type, estimated current and recommended energy, saved energy, last observation time, model name and a human-readable explanation.

The React interface displays current and recommended bars for each lamp, prediction confidence, the selected model, total dimmable energy, estimated energy after recommendation, saving relative to current use and saving relative to a full-brightness baseline.

Chapter 4

Temperature and HVAC Recommendation

4.1 Overview

The temperature subsystem first predicts a temperature persona and then generates an HVAC setpoint. The endpoint `/api/tempreture_recomendation` follows this chain automatically when model predictions are requested. The spelling in the API and filenames is `tempreture_recomendation`; however, the function represents a temperature recommendation.

4.2 Input Sequence

The recommendation service queries the `temperature_data` table for the selected room and lookback interval. The default history window is two hours. The sequence includes floor, facade, room type, room size, outside temperature, indoor temperature, current setpoint, ideal temperature, HVAC mode, occupant state, PIR persona, room state, PIR motion and guest presence.

The final encoded input has 49 features because categorical variables are expanded into one-hot indicators. Two upstream prediction fields are included:

- occupancy prediction: Cleaning, Occupied, Vacant or Unknown; and
- temperature persona prediction: AlwaysOnComfort, EnergySaver, Housekeeping, Preconditioning, Reactive or Unknown.

4.3 Construction of the Training Target

The target setpoint is generated from an explicit comfort and efficiency policy. First, an operating mode is inferred. An existing heating or cooling mode is retained. Otherwise, cooling is selected when the room is at least 0.7°C above the ideal temperature or the outside temperature is at least 27°C . Heating is selected when the room is at least 0.7°C below the ideal temperature or the outside temperature is at most 12°C . All remaining cases use idle mode.

Persona policies define nominal setpoints. For example:

- AlwaysOnComfort: 22.5°C cooling and 22.0°C heating;
- EnergySaver: 26.0°C cooling and 19.0°C heating;
- Reactive: 23.5°C cooling and 21.5°C heating; and
- Housekeeping: 24.0°C cooling and 20.0°C heating.

Vacant rooms use more energy-saving targets: 28 °C for cooling, 17 °C for heating and 25 °C for idle operation.

To prevent an unrealistic change in a single five-minute step, the target is smoothed using the policy target, ideal temperature and current setpoint:

$$T_{\text{target}} = 0.65T_{\text{policy}} + 0.25T_{\text{ideal}} + 0.10T_{\text{current}}.$$

The result is rounded to the nearest 0.5 °C and clipped to the interval from 16 °C to 28 °C.

4.4 Transformer Regressor

The model processes 24 observations, corresponding to two hours of five-minute samples. It uses a 96-dimensional projection, sinusoidal positional encoding, two Transformer encoder layers, four attention heads, a feed-forward dimension of 192, GELU activation and dropout of 0.15.

The last encoded token is passed through layer normalization and a two-layer regression head. A sigmoid limits the normalized output to the interval [0, 1]. It is then transformed back into degrees Celsius:

$$\hat{T} = 16 + 12\hat{y}.$$

Training uses Smooth L1 loss, AdamW, a learning-rate scheduler and chronological training, validation and test partitions. A total of 500,000 sequences were used: 400,000 for training, 50,000 for validation and 50,000 for testing.

4.5 Live Recommendation and Explanation

At serving time, the predicted setpoint is clipped and rounded to the nearest half degree. The backend compares it with the current setpoint and labels the action as *keep*, *raise* or *lower*. It also infers a target mode. In the live implementation, the current HVAC mode is retained when it is heating or cooling; otherwise, room-to-setpoint and outdoor temperature thresholds determine heating, cooling or idle operation.

The API response includes the current setpoint, recommended setpoint, delta, target mode, action, model name, input predictions and a textual reason. The reason explicitly notes when Vacant status permits a more efficient setpoint, when EnergySaver allows a wider comfort band, or when AlwaysOnComfort gives greater priority to comfort.

4.6 Immediate Energy Estimate

The recommendation service estimates the power required by both the current and recommended setpoints using the same HVAC thermal-demand model used by the energy report. For the latest five-minute sample:

$$E_{\text{current}} = P_{\text{current}} \frac{5}{60}, \quad E_{\text{recommended}} = P_{\text{recommended}} \frac{5}{60}.$$

The API returns current and recommended power, current and recommended watt-hours, absolute saving and percentage saving. These values are model-based estimates rather than direct meter readings.

Chapter 5

Energy Consumption Analysis

5.1 Lighting Power Model

Lighting energy is calculated from `EnergyConsumption/DimmerGraphs.xlsx`. The application reads the numeric measurement block and extracts dimmer level, lux, voltage and current for LEDs and bulbs.

LED current is recorded in microamperes, while bulb current is recorded in milliamperes. The implementation therefore uses:

$$P_{\text{LED}} = \frac{V_{\text{LED}} I_{\text{LED}, \mu A}}{10^6},$$

$$P_{\text{bulb}} = \frac{V_{\text{bulb}} I_{\text{bulb}, mA}}{10^3}.$$

The application records levels on a 0–100 scale, but the physical dimmer table uses a 0–254 scale. A recorded application level is converted using:

$$L_{\text{device}} = \text{round} \left(L_{\text{app}} \frac{254}{100} \right).$$

The maximum operational application level is 80. Dimmable LEDs use the power looked up for their converted level. Non-dimmable bulbs use full power whenever their recorded level is greater than zero.

5.2 Lighting Energy Aggregation

The endpoint `/api/energy` receives a room and date interval. The database query selects active lamp records, excludes the artificial `none` location and includes the start timestamp while excluding the end timestamp.

For each five-minute record:

$$E_i = P_i \frac{5}{60}.$$

The system calculates actual energy and a maximum-level baseline for every record. It then aggregates:

- total actual and maximum energy;
- absolute and percentage saving;
- energy grouped by lamp;
- a dimmable-only summary; and
- the number of active samples.

Savings from dimming are calculated separately because non-dimmable bulbs have equal actual and maximum power whenever they are on. The API also evaluates the AI lighting recommendation over every timestamp in the selected interval. This comparison reports recommended energy against a full-brightness dimmable baseline.

5.3 HVAC Thermal-Demand Model

Direct HVAC electricity measurements are not present in the dataset. Consequently, the system estimates HVAC power from thermal demand. The rated limits are 2000 W for heating and 1500 W for cooling.

For heating:

$$\Delta T_{\text{room}} = \max(T_{\text{set}} - T_{\text{room}}, 0),$$

$$\Delta T_{\text{weather}} = \max(T_{\text{set}} - T_{\text{outside}}, 0),$$

$$P_{\text{raw}} = 420 + 260\Delta T_{\text{room}} + 32\Delta T_{\text{weather}}.$$

For cooling:

$$\Delta T_{\text{room}} = \max(T_{\text{room}} - T_{\text{set}}, 0),$$

$$\Delta T_{\text{weather}} = \max(T_{\text{outside}} - T_{\text{set}}, 0),$$

$$P_{\text{raw}} = 380 + 230\Delta T_{\text{room}} + 30\Delta T_{\text{weather}}.$$

Room size is normalized against a reference size of 35 m² and clipped between 0.70 and 1.45. Occupancy factors are 0.55 for Vacant, 0.75 for Cleaning, 0.95 for Occupied without motion, and 1.10 for Occupied with motion. Unknown states use 0.85 without motion and 1.00 with motion.

The final estimate is:

$$P_{\text{HVAC}} = \text{clip}(P_{\text{raw}} F_{\text{size}} F_{\text{occupancy}}, 0, P_{\text{rated}}).$$

Off and unsupported modes produce zero power. Every sample contributes $P_{\text{HVAC}}(5/60)$ watt-hours.

5.4 HVAC Baseline and Reports

The HVAC baseline represents continuous operation at rated power for the dominant active mode. The dominant mode is whichever of heating or cooling appears in more samples. If neither appears, heating is used as a fallback. For N samples:

$$E_{\text{max}} = P_{\text{rated, dominant}} N \frac{5}{60}.$$

The endpoint `/api/hvac.energy` returns total, heating and cooling energy, active and total minutes, mode-level averages, maximum baseline, estimated savings and AI-recommendation energy. The frontend presents actual versus baseline energy, energy by mode, time-share charts and an hourly or daily energy time series.

5.5 Interpretation

Lighting energy uses measured voltage-current relationships and is therefore closer to a physical calculation. HVAC energy is a model-based estimate and should be interpreted as comparative rather than meter-grade consumption. Both analyses assume that one database row represents exactly five minutes. Missing samples or irregular sensor intervals can therefore affect totals.

Chapter 6

Machine Learning Models

6.1 Model Families

The application uses three principal model families:

- Random Forest classifiers for tabular occupancy and lighting persona prediction;
- HistGradientBoosting regression for tabular lighting recommendation; and
- Transformer encoders for sequential occupancy, persona, lighting and temperature tasks.

The classical models provide efficient and interpretable baselines. The Transformers are used when the order and temporal interaction of observations are expected to contain useful information.

6.2 Preprocessing Pipelines

The scikit-learn models save preprocessing and prediction as a single pipeline. Numerical variables are imputed and standardized. Categorical variables are imputed and one-hot encoded with unknown-category handling. Saving the full pipeline ensures that training-time transformations are repeated consistently during API inference.

Transformer inputs use explicit scaling and one-hot feature construction saved in metadata files. The serving code reads the metadata to reconstruct feature order, class labels, sequence length and model dimensions. This is important because changing the order of encoded columns would make a trained neural network invalid even if the same values were supplied.

6.3 Transformer Architecture

All Transformer models follow a related encoder-only design:

1. project each input vector into a hidden representation;
2. add trainable or sinusoidal positional information;
3. process the sequence using multi-head self-attention and feed-forward layers;
4. normalize and pool the sequence or select the final token; and
5. apply a classification or regression head.

Classification models use softmax probabilities and weighted cross-entropy. Regression models produce a normalized scalar that is converted to either a lighting level or HVAC setpoint. AdamW is used throughout the Transformer training scripts. Dropout and weight decay reduce overfitting, while gradient clipping is used in the classification scripts.

6.4 Class Imbalance

Class imbalance is addressed through balanced class weights in Random Forest models and weighted cross-entropy in Transformer classifiers. The class weight for class c is calculated from its training frequency:

$$w_c = \frac{N}{K N_c},$$

where N is the number of training examples, K is the number of classes and N_c is the number of examples belonging to class c .

This increases the penalty for misclassifying rare classes. Nevertheless, the Cleaning occupancy results show that weighting alone cannot fully compensate for very limited examples.

6.5 Training and Data Splitting

Most time-dependent models use chronological splitting. Sequence end times are sorted, and earlier sequences are assigned to training while later sequences are assigned to validation and testing. This is more appropriate than a random split for time-series applications because it simulates prediction on future data.

The lighting persona Random Forest uses a stratified random split because its examples are aggregated room-days rather than direct next-step forecasts. All training scripts use seed 42 to improve reproducibility.

6.6 Evaluation Metrics

Classification models are evaluated with accuracy, precision, recall, F1-score and confusion matrices:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Regression models use mean absolute error, root mean squared error and coefficient of determination:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

The lighting recommendation scripts additionally compare the sum of predicted levels with actual levels. This is an energy proxy rather than a direct watt-hour measurement. The application performs the physical watt-hour calculation separately using the dimmer power table.

6.7 Model Persistence and Runtime Selection

Scikit-learn models are stored with Joblib. Transformer checkpoints are stored as PyTorch `.pt` files accompanied by JSON metadata. The backend caches loaded models and reloads them when the file modification time changes.

The frontend exposes model selectors for occupancy, lighting persona and lighting recommendation. The backend normalizes aliases such as `rf`, `random_forest`, `torch` and `transformer`. Automatic mode selects a Transformer when the expected checkpoint and metadata are available; otherwise it selects the corresponding classical model.

Chapter 7

Results and Evaluation

7.1 Summary of Saved Results

Table 7.1: Evaluation results produced by the project training scripts

Task	Model	Saved test result
Occupancy prediction	Random Forest	Accuracy 98.9%; macro F1 0.692
Occupancy prediction	Transformer	Accuracy 98.8%; macro F1 0.743
Lighting persona	Random Forest	Accuracy 88.5%; macro F1 0.893
Lighting persona	Transformer	Accuracy 92.5%; macro F1 0.926
Temperature persona	Transformer	Accuracy 80.55%; macro F1 0.8848
Lighting recommendation	HistGradientBoosting	MAE 0.105; RMSE 0.356; R^2 0.999
Lighting recommendation	Transformer	MAE 1.027; RMSE 1.880; R^2 0.985
Temperature recommendation	Transformer	MAE 0.217 °C; RMSE 0.310 °C; R^2 0.962

7.2 Occupancy Evaluation

The occupancy models perform very well on Occupied and Vacant observations. The Random Forest achieved F1-scores of 0.997 and 0.978 for these classes, respectively. The Transformer achieved 0.991 and 0.990. The Transformer had a slightly lower overall accuracy but a better macro F1-score because it improved the Cleaning class.

Cleaning precision remains low: 0.057 for the Random Forest and 0.177 for the Transformer. This means many Cleaning predictions are false positives. Cleaning recall is higher, approximately 0.40 for both models, which indicates that class weighting encourages the models to find Cleaning examples but at the cost of precision.

7.3 Persona Evaluation

The lighting persona Transformer outperformed the Random Forest in overall accuracy by four percentage points. Its F1-scores ranged from 0.898 for Balanced to 0.952 for StaticDim. The improvement suggests that complete five-minute daily sequences capture temporal patterns that are partly lost when a day is summarized into aggregated statistics.

The temperature persona model achieved lower overall accuracy. EnergySaver had high recall of 0.8928 but precision of 0.6568, indicating that the model often assigned EnergySaver to observations belonging to another class. Reactive had precision of 0.8291 and recall of 0.7220. These two classes likely overlap because both can produce setpoint changes in response to room conditions.

7.4 Recommendation Evaluation

The HistGradientBoosting lighting model was tested on 2,112,417 rows. Its predicted level-sum proxy was 58,723,973 compared with an efficient target of 58,691,230. The target represented a 27.33% reduction from actual levels, and the model represented a 27.29% reduction.

The lighting Transformer used 500,000 sequences, including 50,000 test sequences. Its model level-sum proxy represented a 29.18% reduction from actual levels, compared with 28.96% for the efficient target. Its errors were larger than those of the tabular model, although an MAE of approximately one level remains small relative to the 0–80 range.

The temperature recommendation Transformer achieved an MAE of 0.217 °C. Since the live application rounds the output to the nearest 0.5 °C, this error is smaller than one control step on average.

7.5 End-to-End Evaluation Considerations

The component metrics evaluate models against labels or policy-derived targets. They do not by themselves prove real guest comfort or building-level energy savings. In live operation, recommendation quality also depends on upstream prediction accuracy. An incorrect occupancy or persona prediction can alter the final lighting or temperature recommendation.

The application partially addresses this issue by returning complete probability distributions and diagnostic features to the user interface. It also permits manual occupancy and persona selection, allowing model-generated and user-selected scenarios to be compared.

Chapter 8

Limitations and Future Improvements

8.1 Sensor Limitations

PIR sensors detect movement rather than continuous physical presence. A guest who is sleeping, reading or otherwise motionless may generate no PIR activity. Conversely, housekeeping or temporary movement may appear similar to guest activity. Future work could combine PIR with door sensors, key-card events, Bluetooth presence, camera-free thermal sensors or CO₂ measurements.

8.2 Class Imbalance and Label Quality

Cleaning has far fewer examples than Occupied and Vacant. The existing class weights improve recall but do not solve low precision. More cleaning examples, explicit housekeeping schedules, focal loss, balanced sequence sampling or a two-stage occupancy classifier could improve this class.

Persona labels are derived from dataset behavior and may not correspond to preferences explicitly stated by real guests. Future studies should collect guest feedback and measure whether a predicted persona remains stable across multiple stays.

8.3 Policy-Derived Recommendation Targets

The lighting and temperature recommendation models learn targets generated by hand-designed efficiency policies. Their high R^2 values therefore indicate that the models reproduce those policies accurately; they do not demonstrate that the policies are globally optimal.

A future system could learn from explicit comfort feedback, room-service requests, manual overrides and measured energy cost. Multi-objective optimization or reinforcement learning could then balance comfort, energy, equipment wear and electricity price.

8.4 Energy Model Limitations

Lighting energy assumes each record covers exactly five minutes and that the dimmer lookup table correctly represents installed hardware. Interpolation is not performed when a converted device level is absent, which can produce missing power values.

HVAC consumption is estimated rather than measured. The thermal model does not directly represent humidity, solar gain, wall insulation, door or window opening, equipment coefficient of performance, fan stages or thermal inertia. Smart plugs or building-management meters should be used to calibrate and validate the power model.

8.5 Data and Generalization

The data covers a limited set of room types, personas, weather conditions and operating periods. Performance may change in another hotel, climate or sensor installation. Cross-hotel testing and seasonal validation are required before general deployment.

The Transformer datasets are capped at 400,000 or 500,000 sequences to control memory use. Although these are large samples, the cap may alter the distribution if sequence construction reaches some rooms or periods earlier than others. Stratified room sampling would provide stronger coverage.

8.6 Runtime and System Improvements

Future implementation work should include:

- real-time ingestion from an IoT message broker;
- authentication and role-based access control;
- model and dataset version tracking;
- uncertainty thresholds that defer low-confidence decisions;
- automatic detection of missing or irregular sensor samples;
- explanations based on feature importance or attention analysis;
- monitoring for data drift and model degradation;
- user override logging and online personalization; and
- direct integration with lighting and HVAC controllers.

8.7 Privacy and Safety

Reservation and behavioral data can reveal occupancy patterns and guest preferences. A production system should minimize personally identifiable information, apply encryption, enforce retention limits and separate identity from behavioral features. Automatic HVAC control should also use hard safety limits and permit staff or guests to override recommendations.

Chapter 9

Conclusion

This project implements a complete smart hotel analytics and recommendation pipeline rather than an isolated visualization or prediction model. The React frontend communicates with a Flask API, which retrieves historical data from SQLite, executes machine-learning models, applies hardware constraints, calculates energy and returns explanations and charts.

Occupancy prediction uses PIR and reservation context to estimate whether a room will be Occupied, Vacant or under Cleaning. Lighting and temperature persona models convert recent behavior into preference categories. These outputs are passed to separate recommendation models that generate per-lamp brightness levels and HVAC setpoints.

The implementation supports both classical tabular models and Transformer sequence models. Random Forest and HistGradientBoosting models provide strong baselines, while Transformers capture temporal behavior over one-hour, two-hour or full-day sequences. The frontend allows model selection and displays probabilities, diagnostics and model names, making the result more transparent than a single unexplained command.

Energy analysis is integrated into the recommendation workflow. Lighting consumption uses measured voltage and current from a dimmer table, with separate handling for dimmable and non-dimmable devices. HVAC consumption is estimated using room temperature, setpoint, outside temperature, room size, occupancy and motion. Both systems compare actual or recommended operation with an explicit baseline.

The saved experiments show high performance for major occupancy classes, strong lighting persona classification and low recommendation errors. The results also reveal important limitations, particularly minority-class occupancy performance, policy-derived targets and the lack of direct HVAC metering.

Overall, the project demonstrates how hotel reservation data, IoT sensor history, machine learning, recommendation policies and energy calculations can be combined into one decision-support application. The next stage should move from offline and simulated evaluation toward calibrated physical measurements, guest feedback, privacy-aware deployment and controlled trials in real hotel rooms.